Covariant Derivatives and Connections in the $\mathcal{UD}$ Calculus

Brian Beckman

Jan 2026

Abstract

In this installment of the $\mathcal{UD}$ series, we address a central issue of differential geometry: how to take a derivative in (or on) a curved manifold without breaking coordinate invariance. Ordinary partial differentiation is not tensorial, producing garbage terms that don't transform properly. The solution is the ***covariant derivative*** and its correction term, the ***connection coefficient*** Γ.

We begin with the non-tensorial transformation law of Γ. Then, applying the constraints of metric compatibility ($\nabla g = 0$) and torsion freedom (symmetry on lower indices of Γ), we derive the particular connection coefficients needed for general relativity, the Christoffel Symbols. This derivation specializes differential geometry to Riemannian geometry as needed for the general relativistic theory of gravitation. Step-by-step derivation in $\mathcal{UD}$ automates tiresome and risky algebraic operations for high assurance in the results.

At this early stage of development, we expose $\mathcal{UD}$ as a script rather than hiding details in a package. This glass-box approach delivers better pedagogy and incremental validation.

Audience

This series of notebooks is targeted at readers with an undergraduate's STEM education—engineering, computer science, mathematics. Exposure to introductory courses in Classical Mechanics (e.g., Goldstein), Electrodynamics (e.g., Jackson), and Relativity (e.g., Schutz) will help.

- *By and large, I do not put standard textbooks in the References section, but offer clickable (non-affiliate!) links instead in the prose of this notebook.*

I also invite readers to "think like a physicist," that is, to demand conceptual cogency, but not always full mathematical rigour.

Transparent Implementation

In developing $\mathcal{UD}$, we made a conscious decision, for the time being, to expose the implementation as a glass-box **script** rather than to hide it in a Package, prioritizing transparency over encapsulation at

this stage of development. When the toolkit is finalized in the future, we'll address encapsulating it in a Package.

Let's load $\mathcal{UD}$'s implementation, the RelativityToolkit, directly into your notebook context.

Load the Code

The code is hosted on GitHub. The implementation, tests, and visualizations are not explained in this notebook. Explanations appear in an earlier notebook. But we run them here, for assurance.

Run the first two cells below. Expect to see test results followed by visualizations and a final gallery result.

- *Note, if you call* **Quit** *in a notebook, evaluation may hang. If you want to "Evaluate Notebook," mark the following cell "Evaluatable" in the Cell→Property menu, call* **Quit** *once, then comment out the* **Quit** *cell, or mark it unevaluatable again. Alternatively, you can quit the kernel via the Evaluation menu (last item). At that point, you can "Evaluate Notebook" in a clean environment. Otherwise, you can evaluate cells in this notebook individually.*

```
(* Cell not evaluatable by default *)
Quit[]
```

In[•]:=
```
(* LOAD THE CODE *)
Module[{base, rules, tests, viz, bust},
 base = "https://raw.githubusercontent.com/rebcabin/RelativityToolkit/main/";
 bust = "?t=" <> ToString[Round[AbsoluteTime[]]];
 (*Force fresh download*)
 (*Define URLs*)
 rules = base <> "RelativityToolkit.wl" <> bust;
 tests = base <> "RelativityToolkit_RegressionTests.wl" <> bust;
 viz = base <> "RelativityToolkit_VisualShowcase.wl" <> bust;
 (*Load them*)
 Get[rules];
 Get[tests];
 Get[viz](*no semicolon on purpose to see the gallery*)]
```

```
================================================================
RUNNING REGRESSION SUITE v1.6.1
================================================================

--- SECTION 1: TYPE CHECKING (VALENCE) ---
[PASS] Vector x^μ ⟶ {{μ}, {}}
[PASS] Covector p_ν ⟶ {{}, {ν}}
[PASS] Mixed Tensor T^μ_ν ⟶ {{μ}, {ν}}
[PASS] Mixed Tensor T_μν ⟶ {{}, {μ, ν}}
```

[PASS] Mixed Tensor $T^{\mu\nu}$ ⟶ {{μ, ν}, {}}

[PASS] Mixed Tensor $T_{\mu}{}^{\nu}$ ⟶ {{ν}, {μ}}

[PASS] Scalar Contraction $x^{\mu}\ p_{\mu}$ ⟶ {{}, {}}

[PASS] Scalar Contraction $p_{\mu}\ x^{\mu}$ ⟶ {{}, {}}

[PASS] Tensor-Vector $T^{\mu}{}_{\nu}\ x^{\nu}$ -> x^{μ} ⟶ {{μ}, {}}

[PASS] Tensor-Vector $T^{\mu}{}_{\nu}\ x^{\mu}$ -> x_{ν} ⟶ {{}, {ν}}

[PASS] Power of Scalar ⟶ {{}, {}}

[PASS] Scalar Multiplication ⟶ {{μ}, {}}

[PASS] Gradient d(f)/dx^{α} -> Down[α] ⟶ {{}, {α}}

Valence Mismatch

» expect error message above {{}, {}}

[PASS] Mismatch Handled Gracefully ⟶ {{}, {}}

--- SECTION 2: ALGEBRAIC CANONICALIZATION ---

[PASS] Dummy Renaming ($A^{\mu}B_{\mu}$ == $A^{\nu}B_{\nu}$) ⟶ $\left(p_{\ i1}\right)\left(x^{\ i1}\right)$

[PASS] Commutativity ($A^{\mu}B_{\mu}$ == $B_{\nu}A^{\nu}$) ⟶ $\left(p_{\ i1}\right)\left(x^{\ i1}\right)$

[PASS] Index Coalescing ($A^{\mu}B_{\mu}$ + $A^{\nu}B_{\nu}$ == 2 $A^{\rho}B_{\rho}$) ⟶ $2\left(p_{\ i1}\right)\left(x^{\ i1}\right)$

[PASS] Double Dummy Pairs ($P^{\mu}\ Q_{\mu}\ S^{\nu}\ T_{\nu}$) ⟶ $\left(P^{\ i1}\right)\left(Q_{\ i1}\right)\left(S^{\ i2}\right)\left(T_{\ i2}\right)$

[PASS] Free Indices Preserved ⟶ $\left\{\left(A^{\ \alpha}\right)\left(B^{\ \beta}\right), \left(A^{\ i1}\right)\left(B^{\ i1}\right)\right\}$

--- SECTION 3: PHYSICS, METRIC & TRANSFORMATIONS ---

[PASS] Lowering Index ($g_{\mu\nu}\ A^{\nu}$ → A_{μ}) ⟶ $A_{\ \mu}$

[PASS] Raising Index ($g^{\mu\nu}\ p_{\nu}$ → p^{μ}) ⟶ $p^{\ \mu}$

[PASS] Inverse Metric ($g^{\mu\alpha}\ g_{\alpha\nu}$ → δ^{μ}_{ν}) ⟶ δ^{μ}_{ν}

[PASS] Alpha-Conversion: distinct indices) ⟶ {μ11, μ12}

[PASS] Triple Metric Sandwich (g.g.g -> g) ⟶ $g_{\ \mu\ \nu}$

[PASS] Delta-Gamma Contraction ⟶ $\Gamma^{s}_{a\,b}$

--- SECTION 4: METRIC EQUIVALENCE ($A^{\mu}\ B_{\mu}$ == $A_{\nu}\ B^{\nu}$) ---

[PASS] Structural Distinction $A^{\mu}\ B_{\mu}$!= $A_{\nu}\ B^{\nu}$ initially ⟶ $\left\{\left(A^{\ i1}\right)\left(B_{\ i1}\right), \left(A_{\ i1}\right)\left(B^{\ i1}\right)\right\}$

𝒰𝒟 will rearrange terms

[PASS] Metric Equivalence ($A_{\nu}\ B^{\nu}$ reduces to $A^{\mu}\ B_{\mu}$) ⟶ $\left(A^{\ i1}\right)\left(B_{\ i1}\right)$

--- SECTION 5: SECOND DERIVATIVE Box Structure ---

[PASS] Found FractionBox + ^2 ⟶ True

--- SECTION 6: CALCULUS ENGINE ---

[PASS] Leibniz Product Rule ⟶ $g\, f_{,\mu} + f\, g_{,\mu}$

[PASS] Covariant Derivative Expansion ⟶ True

[PASS] Differentiation Linearity ⟶ True

[PASS] CD of Scalar reduces to Partial ⟶ $phi\$8017_{,\mu}$

[PASS] Scalar CD contains no Connection Coefficients ⟶ True

[PASS] Nested CD Alpha-Conversion (Unique Indices Generated) ⟶ True

--- SECTION 7: QUOTIENT THEOREM (Extraction) ---

[PASS] Quotient Extraction (Simple) ⟶ T_{s}

[PASS] Quotient Extraction (Distributed Dummies) ⟶ $R_{s} + S_{s}$

[PASS] Quotient Extraction (Zero Case) ⟶ 0

--- SECTION 8: TENSOR FORM ROBUSTNESS ---

[PASS] TensorForm maps arbitrary formals (Q, Z, I99) to Greek ⟶ True

--- SECTION 9: COMMA NOTATION DISPLAY LOGIC ---

[PASS] Atomic Tensor ⟶ $A^{\mu}{}_{,\nu}$

[PASS] Sealed Product (Gamma Glue) ⟶ $((A^{\mu})\, \Gamma^{a}_{b\,c})_{,\nu}$

[PASS] Sealed Sum ⟶ $(A + B)_{,\nu}$

[PASS] Sealed Power ⟶ $\left(A^{2}\right)_{,\nu}$

[PASS] Arbitrary Function (Sin) ⟶ $Sin[theta]_{,\nu}$

[PASS] Nested Math (A*(B+C)) ⟶ $(A\,(B + C))_{,\nu}$

REGRESSION SUITE COMPLETE

PASSED: 44

FAILED: 0

STATUS: GREEN (Ready for Publication)

==========

VISUAL SHOWCASE: FORMAL DIFFERENTIAL GEOMETRY

v1.6.1

==========

1. The Tensor Zoo (Standard Objects)

These objects carry valence metadata hidden inside their structure.

Object Name	Wolfram Input	Textbook Output
Contravariant Vector	x[$\mathcal{U}[\mu]$]	x^{μ}
Covariant Covector	p[$\mathcal{D}[\nu]$]	p_{ν}
Mixed Tensor	T[$\mathcal{U}[\mu]$, $\mathcal{D}[\nu]$]	$T^{\mu}{}_{\nu}$
The Metric	g[$\mathcal{D}[\mu]$, $\mathcal{D}[\nu]$]	$g_{\mu\nu}$
Inverse Metric	g[$\mathcal{U}[\mu]$, $\mathcal{U}[\nu]$]	$g^{\mu\nu}$
Kronecker Delta	δ[$\mathcal{U}[\mu]$, $\mathcal{D}[\nu]$]	δ^{μ}_{ν}
Jacobian (Partial)	Partials[x[$\mathcal{U}[\alpha]$], x[$\mathcal{U}[\beta]$]]	$\frac{\partial x^{\alpha}}{\partial x^{\beta}}$

2. Metric Gymnastics (Raising & Lowering)

Watch the metric 'swallow' indices to change their variance.

Index Lowering

Input: $(A^{\nu})(g_{\mu\nu})$

Result: A_{μ}

Index Raising

Input: $(g^{\mu\nu})(p_{\nu})$

Result: p^{μ}

The Inverse Identity

Input: $(g_{\alpha\nu})(g^{\mu\alpha})$

Result: δ^{μ}_{ν}

3. Coordinate Transformations (Alpha-Conversion)

Note the automatically generated unique dummy index (e.g., $\mu$$...).

Transforming a Vector

Input: $A^{a'}$

Result: $\left(A^{\mu 20}\right)\frac{\partial x^{a'}}{\partial x^{\mu 20}}$

4. Canonicalization & Pretty Printing

Simplifying dummy indices into standard Greek letters.

Raw Sum (Different Indices): $(A^{\mu})(B_{\mu}) + (A^{\nu})(B_{\nu})$

Canonicalized (TensorForm): $2\left(A^{\lambda}\right)(B_{\lambda})$

5. Physical Equivalence Proof

Demonstrating that $A^{\mu}\ B_{\mu}$ is equivalent to $A_{\nu}\ B^{\nu}$.

(𝒰𝒟 will reorganize and canonicalize displays.)

Step	Expression	Input
1. Start	(A $_{\nu}$) (B $^{\nu}$)	Input: A[𝒟[ν]]B[𝒰[ν]]
2. Definition	(A $^{\alpha}$) (B $^{\nu}$) (g $_{\nu\ \alpha}$)	(g[𝒟[ν],𝒟[α]] A[𝒰[α]]) B[𝒰[ν]]
3. Commute?	(A $^{\alpha}$) (B $^{\nu}$) (g $_{\nu\ \alpha}$)	A[𝒰[α]] (g[𝒟[ν],𝒟[α]] B[𝒰[ν]])
4. Contract	(A $_{\nu}$) (B $^{\nu}$)	step2Term/. metricRules
5. Equality?	True	lhs:4 === rhs:1

==========

--- Gallery of Derivatives ---

Out[•]=

Description	Rendered Output
Generic Mixed Partial	$\frac{\partial^2 f}{\partial x\,\partial y}$
Coordinate Hessian	$\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta}\,\partial x^{\alpha}}$
Atomic Tensor	$A^{\mu}{}_{,\nu}$
Sealed Product (v.1.6.1 fix)	$\left((A^{\mu})\,\Gamma^{\rho}_{\sigma\lambda}\right)_{,\nu}$
Sealed Sum (Linearity)	$(A^{\mu}+B^{\mu})_{,\nu}$
Sealed Power (Non-Linear)	$\left(\phi^2\right)_{,\nu}$
Covariant Derivative (Abstract)	$A^{\mu}{}_{;\nu}$
Covariant Derivative (Expanded)	$A^{\mu}{}_{,\nu}+\left(A^{\lambda}\right)\Gamma^{\mu}_{\nu\lambda}$

Covariant Derivative

- *A .wl block-version of code for this section, minus the prose, is available at https://github.com/rebcabin/RelativityToolkit/blob/main/covariant_derivative.wl.*

We Want a Derivative!

Consider a curve with invertible, continuous, differentiable coordinate functions $x^{\alpha}(\lambda)$ along the curve, and tangent (contravariant) vectors $u^{\alpha} \equiv dx^{\alpha}/d\lambda$. Also consider a (contravariant) vector field $A^{\alpha}\left(x^{\beta}(\lambda)\right)$

defined in neighborhoods around every point $x^\alpha(\lambda)$ along the curve. We'd like a "derivative" of the vector field, schematically:

$$\frac{dA^\alpha}{dx^\beta} \stackrel{?}{\equiv} \frac{A^\alpha\left(x^\beta + dx^\beta\right) - A^\alpha\left(x^\beta\right)}{dx^\beta} \tag{1}$$

- *We'll have a better, "cool" notation for $\frac{dA^\alpha}{dx^\beta}$ soon enough. For now, read it as "the derivative we're trying to define."*

Seems a reasonable thing to ask: "How much does component α of A change when I change coordinate β a little?"

However, Equation 1 is not a cogent definition of derivative on a manifold. We cannot subtract vectors at different points, not even infinitesimally different points. The coordinate axes at $\left(x^\beta + dx^\beta\right)$ don't point in the same directions as the coordinate axes at x^β because the manifold bends and twists away between the two points! The components of A are not commensurable at the two points because the two vectors are in **different tangent spaces**. Subtracting components as prescribed by Equation 1, numerically, scrambles information from other directions.

Parallel Transport

To answer this conundrum, we need backwards ***parallel transport***, to move the vector $A^\alpha\left(x^\beta + dx^\beta\right)$ smoothly and with a ***provable minimum*** of bending and twisting, back from $x^\beta + dx^\beta$ to the point x^β, so we can measure the components of the transported $A^\alpha\left(x^\beta + dx^\beta\right)$ on the coordinate axes at point x^β and then subtract the numbers.

$$\frac{dA^\alpha}{dx^\beta} \stackrel{?}{\equiv} \frac{\text{ParallelTransport}\left(A^\alpha\left(x^\beta + dx^\beta\right),\ \text{to } x^\beta\right) - A^\alpha\left(x^\beta\right)}{dx^\beta} \tag{2}$$

Coordinate-Independence

Now, with subtraction conceptually solved, how will that difference survive a coordinate transformation? How can I find out, when I **change coordinates from x^β to $x^{\beta'}$ at the same point**, how much component $\alpha \to \alpha'$ of A changes when I change coordinate $\beta \to \beta'$? That is, what's the value of $\frac{dA^{\alpha'}}{dx^{\beta'}}$ when all I know is $\frac{dA^\alpha}{dx^\beta}$?

To answer this conundrum, we insist, as always, that the *intrinsic* geometry and physics **not** depend on choice of coordinates. But what does *intrinsic* mean when all we have are numerical components whose values depend on choice of coordinates?

Intrinsic Means Tensorial

Tensorial quantities encapsulate both intrinsic and coordinate-dependent information. If our desired, derivative-like quantities $\frac{dA^\alpha}{dx^\beta}$ transform like a (1, 1) tensor when we change the basis vectors at point β, we'll capture both:

- *intrinsic* geometric and physical properties of a tensor

- *extrinsic* artifacts arising from choice of coordinates
  - Think of changing from Cartesian to spherical coordinates in flat Euclidean 3-space: the numbers change but the geometry and physics don't.

Plus, we will be able to separate intrinsic from extrinsic quantities cleanly.

- *The first couple of chapters of Lovelock & Rund address this issue of intrinsic, geometry-dependent effects versus extrinsic, coordinate-dependent effects rigorously. Thinking like physicists, let's be satisfied with the conceptual description above for now.*

Tensorial and Linear Requirements

Agree that the components of our desired derivative, $\frac{dA^\alpha}{dx^\beta}$, change with coordinate frames in exactly the way components of any other tensor with one up and one down index, would change. These components are projections of the vector A on the basis vectors of the coordinate system x^β, and measure both intrinsic and extrinsic information.

It turns out that simply demanding that our new derivative be **tensorial and linear** suffices to solve all the problems above, including optimality.

- *TODO: This claim of optimality may not be good enough, even for a physicist happy with mathematical hand-waving. Misner, Thorne, and Wheeler [MTW] devote many chapters (8-15) to a more careful, physicist's level of understanding, including demonstrations that the covariant derivative perform parallel transport! Those are missing pieces in this notebook.*
- *A mathematician will demand rigorous proofs. For those, see the many texts on differential geometry, perhaps starting with do Carmo's Differential Geometry of Curves and Surfaces. Other texts such as Tristan Needham, Visual Differential Geometry and Forms, and Michael Spivak's 5-volume Comprehensive Introduction to Differential Geometry are recommended.*

The Partials are Not Tensors

First, let's check that $A^\alpha{}_{,\beta}$ is not tensorial. It's not enough for our desired derivative, but we'll figure out how to correct it.

- *The comma notation, $A^\alpha{}_{,\beta}$, is short for $\frac{\partial A^\alpha}{\partial x^\beta}$, also written $\partial_\beta A^\alpha$. Though we don't yet know how to define $\frac{dA^\alpha}{dx^\beta}$, $\frac{\partial A^\alpha}{\partial x^\beta}$ is just good-ol' partial derivative from ordinary calculus, a mechanical, symbolical calculation that Wolfram automates very well.*
- ***𝒰𝒟 embraces the comma notation** as you can see from the Visual Showcase gallery above.*

Let's transform bases and calculate $A^{\alpha'}$, components in basis $x^{\alpha'}$, from A^α, components in basis x^α. A is a contravariant tensor, so transforms as follows:

$$A^{\alpha'} \equiv \frac{\partial x^{\alpha'}}{\partial x^\alpha} A^\alpha \tag{3}$$

- *A mnemonic for "contravariant:" let $\frac{\partial x^{\alpha'}}{\partial x^\alpha}$ measure feet (primed) per inch (unprimed), that is (1/12). Any unprimed contravariant vector A^α is, mnemonically, like a velocity in inches per second, and its primed partner $A^{\alpha'}$ is, mnemonically, like a velocity in feet per second. Multiply A^α (inches per second) by $\frac{\partial x^{\alpha'}}{\partial x^\alpha}$*

(feet per inch, 1/12*) to get* $A^{\alpha'}$ *(feet per second), i.e.,* $A^{\alpha'} = \frac{\partial x^{\alpha'}}{\partial x^{\alpha}} A^{\alpha}$*. The same rule goes for any kind of coordinate change, curvilinear, non-orthogonal, whatever: Contravariant? Multiply by* $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$*.*

- *Another mnemonic: the unprimed up index of* A^{α} *matches* contrariwise *the unprimed down index of* $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$*, given that an up index in the denominator* ∂x^{α} *of* $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$ *is a down index of the whole* $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$*.* $\mathcal{UD}$ *automates this* ***valence check****.*
- *The quantities* $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}$ *are the* ***Jacobian of the coordinate transformation*** *from* x^{α} *to* $x^{\alpha'}$*, sometimes written (totally ambiguously!) as J, with* $\frac{\partial x^{\alpha}}{\partial x^{\alpha'}}$ *being* J^{-1}*. In manual calculations, one must constantly check the "direction" of Jacobian every time; it's a fertile source of error.* ***$\mathcal{UD}$ won't let you make this mistake!***
- *Covariant tensors transform with the matching index co-wise in the numerator of the Jacobian. The mnemonical example is a gradient, say degrees of temperature per foot or inch.*

Now calculate partial derivatives with respect to the primed frame:

$$A^{\alpha'}{}_{,\beta'} \equiv \frac{\partial}{\partial x^{\beta'}} \left(\frac{\partial x^{\alpha'}}{\partial x^{\alpha}} A^{\alpha} \right) \tag{4}$$

- *This notation is different from Lovelock & Rund, who write* $\overline{x}^{\alpha}$ *for the transformation functions, with Greek indices that have no primes. However, we will not be confused:* $x^{\alpha'}$ *is short for* $x^{\alpha'}(x^{\beta})$*, four functions of four arguments each in relativistic physics, N functions of N arguments more generally. These functions express new coordinates in terms of old. The prime isn't really on the index, but on the pair of literal x and the index. We write it on the index only for ease of typesetting. Also be reminded that the partial derivatives with respect to independent variables* x^{β} *are mechanical, symbolical operations.*

Let's have $\mathcal{UD}$ do this bit of calculus. As stated, the code is maintained as a block of code rather than as alternating prose and code cells. But the comments and the Echoes tell the whole story as prose. **Read The Code** and the results.

In[•]:=
```
ClearAll[ap, bp, mu];

(*Step 1: Define the Input*)
(*Partial derivative of the Transformed Vector A' w.r.t x'*)
(*Input: via Partials*)
inputTerm = Echo[
   Partials[
    (*Expand A^α' into Jac * A^μ immediately via existing rule*)
    A[𝒰[ap]] /. robustTransformRules, x[𝒰[bp]]],
   "1. Input ∂_β'(A^α') ≡ A^α',_β':"];

(*Step 2:Mechanical Expansion*)
(*Apply Leibniz and Chain Rule repeatedly until stable*)
expandedTerm = Echo[ExpandDerivatives[inputTerm],
   "2. Mechanically Expanded Derivative:"];

(*Step 3:Formatting*)
(*Map internal unique dummies to readable Greek*)
prettyResult = Echo[expandedTerm /. {ap → α', bp → β', mu → μ},
   "3. Peek at unique dummy indices"];

brokenPartial = Echo[TensorForm[prettyResult],
   "4. Expanded ∂_β'(A^α') in Readable Greek"];
```

» 1. Input $\partial_{\beta'}(A^{\alpha'}) \equiv A^{\alpha'}{}_{,\beta'}$: $\left(\left(A^{\mu 23}\right)\frac{\partial X^{ap}}{\partial X^{\mu 23}}\right)_{,bp}$

» 2. Mechanically Expanded Derivative: $\left(A^{\mu 23}\right)\frac{\partial^2 X^{ap}}{\partial X^{\mu 23}\partial X^{bp}} + \left(A^{\mu 23}{}_{,bp}\right)\frac{\partial X^{ap}}{\partial X^{\mu 23}}$

» 3. Peek at unique dummy indices $\left(A^{\mu 23}\right)\frac{\partial^2 X^{\alpha'}}{\partial X^{\beta'}\partial X^{\mu 23}} + \left(A^{\mu 23}{}_{,\beta'}\right)\frac{\partial X^{\alpha'}}{\partial X^{\mu 23}}$

» 4. Expanded $\partial_{\beta'}(A^{\alpha'})$ in Readable Greek $\left(A^{\lambda}\right)\frac{\partial^2 X^{\alpha'}}{\partial X^{\beta'}\partial X^{\lambda}} + \left(A^{\lambda}{}_{,\beta'}\right)\frac{\partial X^{\alpha'}}{\partial X^{\lambda}}$

Connection Coefficients

- *A .wl block-version of code for this section, minus the prose, is available at https://github.com/rebcabin/RelativityToolkit/blob/main/law_of_gamma.wl.*

The Partials are a Mess

The partial derivative $A^{\alpha}{}_{,\beta}$ is not tensorial, else it would transform like $\frac{\partial x^{\alpha'}}{\partial x^{\alpha}}\frac{\partial x^{\beta}}{\partial x^{\beta'}}A^{\alpha}{}_{,\beta}$. When we differentiate $A^{\alpha'}$, the transform of A^{α}, in the new coordinate system $x^{\beta'}$ (from Echo 1, above):

$$\frac{\partial}{\partial x^{\beta'}}\left(A^{\alpha'} \equiv A^{\mu}\frac{\partial x^{\alpha'}}{\partial x^{\mu}}\right) \equiv A^{\alpha'}{}_{,\beta'} \equiv \left(A^{\mu}\frac{\partial x^{\alpha'}}{\partial x^{\mu}}\right)_{,\beta'} \tag{5}$$

we don't get the transform $\left(\frac{\partial x^{\alpha}}{\partial x^{\alpha'}}\frac{\partial x^{\beta'}}{\partial x^{\beta}}A^{\alpha'}{}_{,\beta'}\right)$ of the derivative $A^{\alpha'}{}_{,\beta'}$, but two terms that are difficult to interpret: $\left(A^{\lambda}\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\partial x^{\lambda}}\right)$ and $\left(A^{\lambda}{}_{,\beta'}\frac{\partial x^{\alpha'}}{\partial x^{\lambda}}\right)$ from Echo 4 above).

These terms have factors, $\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\partial x^{\lambda}}$ and $A^{\lambda}{}_{,\beta'}$ respectively, that are calculated in both coordinate systems, and that's disturbing. We can extract one Jacobian factor of $\frac{\partial x^{\beta}}{\partial x^{\beta'}}$, but that's it. Rewrite the last line, Echo 4, as follows:

$$\begin{aligned} &A^{\alpha'}{}_{,\beta'} = A^{\lambda}\left(\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'}\partial x^{\lambda}}\right) + \left(A^{\lambda}{}_{,\beta'}\frac{\partial x^{\alpha'}}{\partial x^{\lambda}}\right) = \\ &A^{\lambda}\frac{\partial x^{\beta}}{\partial x^{\beta'}}\left(\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta}\partial x^{\lambda}}\right) + \left(A^{\lambda}{}_{,\beta}\frac{\partial x^{\beta}}{\partial x^{\beta'}}\right)\frac{\partial x^{\alpha'}}{\partial x^{\lambda}} = \frac{\partial x^{\beta}}{\partial x^{\beta'}}\left(A^{\lambda}\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta}\partial x^{\lambda}} + A^{\lambda}{}_{,\beta}\frac{\partial x^{\alpha'}}{\partial x^{\lambda}}\right) \end{aligned} \tag{6}$$

The quantity $\frac{\partial x^{\beta}}{\partial x^{\beta'}}\left(A^{\lambda}\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta}\partial x^{\lambda}} + A^{\lambda}{}_{,\beta}\frac{\partial x^{\alpha'}}{\partial x^{\lambda}}\right)$ is not tensorial because it has a free index α'. Worse, it depends on both coordinate systems, the primed and the unprimed, and we can't have that! Worse still, $\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta}\partial x^{\lambda}}$ the ***Hessian*** of the coordinate transformation, is not a geometrical object, but rather an algebraic object. It can't itself be transformed. It depends on particular choices of two coordinate systems. It's of no help.

The Way Out: Covariant Derivative

There's a way out: insist on a certain form. Write the tensorial derivative we want, $\frac{dA^{\alpha}}{dx^{\beta}}$, as the non-tensorial partial derivative $A^{\alpha}{}_{,\beta}$ plus a correction. Demand that the correction be linear in A^{α} and that the sum—partial derivative plus correction—transform tensorially. Defining the ***semicolon notation*** for this "corrected derivative" $A^{\alpha}{}_{;\beta}$

$$A^{\alpha}{}_{;\beta} \equiv A^{\alpha}{}_{,\beta} + \Gamma^{\alpha}{}_{\beta\mu}A^{\mu} \tag{7}$$

require that it transform as follows:

$$A^{\alpha'}{}_{;\beta'} = \frac{\partial x^{\alpha'}}{\partial x^{\alpha}}\frac{\partial x^{\beta}}{\partial x^{\beta'}}A^{\alpha}{}_{;\beta} \tag{8}$$

then solve for $\Gamma^{\alpha}{}_{\beta\mu}$. Notice contraction on the third index: $\Gamma^{\alpha}{}_{\beta\mu}A^{\mu}$.

$A^{\alpha}{}_{;\beta}$ is called the ***covariant derivative***.

Despite the name, the covariant derivative is *contravariant* in α and *covariant* in β, as we see shortly below. The word "covariant" here does not have its usual meaning, but we're stuck with this historical nomenclature.

- *The form of a term plus a correction is ubiquitous in linear theories such as that for the Kalman filter (see my notebook Kalman filtering as a functional fold).*
- *The contravariant vector A is tensorial, measuring intrinsic properties of A. Γ is not tensorial, mixing in extrinsic information about the coordinate system. The product A.Γ is tensorial again.*

- *If we do our job right (proving tensoriality and linearity), $A^{\alpha}{}_{;\beta}$ becomes the promised "cool notation" for $\frac{dA^{\alpha}}{dx^{\beta}}$, the derivative we want, the covariant derivative.*
- *One may also write $\partial_{\beta} A^{\alpha}$ for $A^{\alpha}{}_{,\beta}$ and $\nabla_{\beta} A^{\alpha}$ for $A^{\alpha}{}_{;\beta}$. These forms frequently appear in the literature.*
- *TODO: Why must the correction $\Gamma^{\alpha}{}_{\beta\mu} A^{\mu}$ be linear in A?*
- *SPOILER: We want our new derivative ∇ (aka the semicolon) to behave like any other derivative. That means it must satisfy two properties:*
  - *Linearity: $\nabla_{\mu}(A^{\nu} + B^{\nu}) = \nabla_{\mu} A^{\nu} + \nabla_{\mu} B^{\nu}$*
  - *Leibniz Rule: $\nabla_{\mu}(f A^{\nu}) = A^{\nu} \partial_{\mu} f + f \nabla_{\mu} A^{\nu}$*

  If we assume the general form of derivative plus a correction, $\nabla_{\mu} A^{\nu} = \partial_{\mu} A^{\nu} + C^{\nu}(A, \partial A, \partial^2 A \ldots)$, then:
  - *Linearity forces $C(A)$ to be linear in A.*
  - *Leibniz forces $C(A)$ to have no derivatives of A.*

  Therefore, the correction must be a multiplicative term: matrix multiplication of A by some object Γ.
- *Given what we want to measure, we see why $\Gamma^{\alpha}{}_{\beta\mu}$ **must have three indices**: it measures how much A^{α} changes when we change x^{β} a little, in terms of the components of A^{μ} contracted against $\Gamma^{\alpha}{}_{\beta\mu}$ at the home point x^{μ}! Parallel transport simply must look exactly like this, no more, no less. (a physicist's hand-wave; mathematicians want more!)*
- *TODO: Is $\Gamma^{\alpha}{}_{\beta\mu}$ symmetric in β and μ? HINT: don't rathole on this now!*

Connection Coefficients

In this context, $\Gamma^{\alpha}{}_{\beta\mu}$ are called ***connection coefficients***. Later, we'll use the same symbol $\Gamma^{\alpha}{}_{\beta\mu}$ for Christoffel Symbols, which are particular connection coefficients, those that satisfy restrictive physical requirements for general relativity.

- *In restricting connection coefficients for gravitation in general relativity, we're saying "only certain kinds of manifold geometries and topologies are useful for describing gravitation." Connection coefficients are more general than Christoffel symbols, pertaining to a broader class of manifolds.*

Covariant Derivatives and Connections in $\mathcal{UD}$

$\mathcal{UD}$'s CD functions shows off the evaluated covariant derivative:

```
In[•]:= "A^α';β'" == TensorForm[CD[A[𝒰[α]], x[𝒰[β]]]]
```

```
Out[•]=
A^α'_;β' == A^α_,β + (A^λ) Γ^α_βλ
```

`CD`'s held form exhibits semicolon notation!

```
In[•]:= Hold[CD[A[𝒰[α]], x[𝒰[β]]]]
```

```
Out[•]=
Hold[A^α_;β]
```

Solving for Connection Coefficients $\Gamma^{\alpha}{}_{\beta\mu}$

Let's have $\mathcal{UD}$ do all the dirty work. We're not only going to solve for $\Gamma^{\alpha}{}_{\beta\mu}$, assuming only that $A^{\alpha}{}_{;\beta} \equiv (A^{\alpha}{}_{,\beta} + A^{\mu}\,\Gamma^{\alpha}{}_{\beta\mu})$ is tensorial (already encoded in $\mathcal{UD}$'s `CD`), but we'll derive the transformation law for $\Gamma^{\alpha}{}_{\beta\mu}$, proving that Γ is not tensorial, and show how the contraction $A^{\mu}\,\Gamma^{\alpha}{}_{\beta\mu}$ subtracts, exactly, the non-tensorial parts of the messy partial derivative $A^{\alpha}{}_{,\beta}$.

As usual, **Read The Code** and the results. Note specifically how TensorForm does the index coalescing and simplification, right where humans go down the drain!

IMPORTANT: The final step invokes a Quotient Theorem, by which, schematically, $(T'\,\Gamma^{\alpha'}{}_{\beta'\mu'} = \langle Jacobians\rangle * T\,\Gamma^{\alpha}{}_{\beta\mu})$ implies $(\Gamma^{\alpha'}{}_{\beta'\mu'} = \langle Jacobians\rangle * \Gamma^{\alpha}{}_{\beta\mu})$, factoring out (quotienting) the tensors T and T' and their Jacobians, if and only if (iff) they're truly arbitrary.

- *Lovelock & Rund write dire warnings about these theorems, that misapplication is the occasion of many published errors. Here, because we know the answer is correct, even by comparison to Lovelock & Rund, we're going to hand-wave the issue, here.*

In[ ]:=
```
(* ========================================================================== *)
(* DERIVATION:The Transformation Law of Gamma *)
(* ========================================================================== *)
ClearAll[ap, bp, mu, nu, manualChainRule, allRules, targetTensor, Aprimed, xprimed];

(* 0. SPECIAL RULE, not in 𝒰𝒟 proper *)
(* A,β' → ∂x^μ/∂x^β' A,μ *)
(* Condition/;!MatchQ[expr,x[_]] prevents recursing a Jacobian on tself *)
manualChainRule =
  Partials[expr_, x[𝒰[bp]]] /; ! MatchQ[expr, x[_]] :>
   Module[{fresh = Unique["μ"]},
    Partials[x[𝒰[fresh]], x[𝒰[bp]]] *
     Partials[expr, x[𝒰[fresh]]]];

(* Combine differentiation Rules from 𝒰𝒟
 and manualChainRule here into a single flat list *)
allRules = Flatten[{differentiationRules, manualChainRule}];

(* 1. transformed covariant derivative *)
targetTensor = Echo[
   (*Jacobians*)
   Partials[x[𝒰[ap]], x[𝒰[mu]]] *
    Partials[x[𝒰[nu]], x[𝒰[bp]]] *
    CD[A[𝒰[mu]], x[𝒰[nu]]],
   "1. 𝒰𝒟 says A^α'_;β' ="];

(* 2. just calculate *)
```

```
(* Define A' and x' *)
Echo[Aprimed = A[𝒰[ap]] /. robustTransformRules, "2a. A^α' ="];
xprimed = x[𝒰[bp]];

(* A^μ' Γ^α'_β' μ' = A^α'_;β' - A^α'_,β' *)
brokenPartialOnly = Echo[
   Partials[Aprimed, xprimed] //. allRules,
   "2b. Broken partial A^α'_,β' ="];
gammaPrimeTimesVector = targetTensor - brokenPartialOnly;

(* 3. FORMATTING (Lovelock Style) *)
(* Collect terms by A to isolate the transformation law *)
prettyResult = Echo[
   gammaPrimeTimesVector /. {ap → α', bp → β', mu → μ, nu → ν},
   "3a. Residual (A^α'_;β' - A^α'_,β') ="];

Print["\n=== Γ TRANSFORMATION LAW (without quotient) ==="];
Print["The term involving Γ' must satisfy:"];
(* HERE IS THE MAGIC *)
(* Tensor Form does index coalescing and simplification! *)
Echo[
  TensorForm[Collect[prettyResult, A[𝒰[_]]]],
  "4. A^γ' Γ^α'_β'γ' = "];

(* 4.APPLY QUOTIENT THEOREM *)
(* Extract the coefficient of A^cp=A[𝒰[cp]]. *)

(* temporarily force index cp *)
inverseVectorRule = A[𝒰[idx_]] :→
   Partials[x[𝒰[idx]], x[𝒰[cp]]] * A[𝒰[cp]];

(* Apply to residual *)
(* ensuring every A term is linear in A^cp *)
rhsWithTargetA = gammaPrimeTimesVector /. inverseVectorRule;

(* Extract (strip) the coefficient *)
finalGammaLaw = Coefficient[rhsWithTargetA, A[𝒰[cp]]];

(* 5. FINAL FORMATTING *)
prettyResult =
  finalGammaLaw /. {ap → α', bp → β', cp → γ', mu → μ, nu → ν};

Print["\n=== Γ TRANSFORMATION LAW ==="];
```

```
Print["By a Quotient Theorem, stripping A to find Γ':"];

(* MORE MAGIC FROM TensorForm *)
temp$ = Echo[TensorForm[prettyResult], "5. Γ^α'_β'γ' ="];
Print["Not Tensorial!"]
```

» 1. $\mathcal{UD}$ says $A^{\alpha'}{}_{;\beta'} = \frac{\partial x^{ap}}{\partial x^{mu}} \frac{\partial x^{nu}}{\partial x^{bp}} \left(A^{mu}{}_{,nu} + \left(A^{\lambda 26}\right) \Gamma^{mu}_{nu\,\lambda 26}\right)$

» 2a. $A^{\alpha'} = \left(A^{\mu 27}\right) \frac{\partial x^{ap}}{\partial x^{\mu 27}}$

» 2b. Broken partial $A^{\alpha'}{}_{,\beta'} = \left(A^{\mu 27}\right) \frac{\partial^2 x^{ap}}{\partial x^{\mu 27} \partial x^{bp}} + \left(A^{\mu 27}{}_{,\mu 28}\right) \frac{\partial x^{ap}}{\partial x^{\mu 27}} \frac{\partial x^{\mu 28}}{\partial x^{bp}}$

» 3a. Residual $(A^{\alpha'}{}_{;\beta'} - A^{\alpha'}{}_{,\beta'}) = -\left(A^{\mu 27}\right) \frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'} \partial x^{\mu 27}} -$

$$\left(A^{\mu 27}{}_{,\mu 28}\right) \frac{\partial x^{\mu 28}}{\partial x^{\beta'}} \frac{\partial x^{\alpha'}}{\partial x^{\mu 27}} + \frac{\partial x^{\nu}}{\partial x^{\beta'}} \frac{\partial x^{\alpha'}}{\partial x^{\mu}} \left(A^{\mu}{}_{,\nu} + \left(A^{\lambda 26}\right) \Gamma^{\mu}_{\nu\,\lambda 26}\right)$$

```
=== Γ TRANSFORMATION LAW (without quotient) ===
The term involving Γ' must satisfy:
```

» 4. $A^{\gamma'} \Gamma^{\alpha'}{}_{\beta'\gamma'} = -\left(A^{\lambda}\right) \frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'} \partial x^{\lambda}} + \left(A^{\lambda}\right) \frac{\partial x^{\kappa}}{\partial x^{\beta'}} \frac{\partial x^{\alpha'}}{\partial x^{\rho}} \Gamma^{\rho}_{\kappa\,\lambda}$

```
=== Γ TRANSFORMATION LAW ===
By a Quotient Theorem, stripping A to find Γ':
```

» 5. $\Gamma^{\alpha'}{}_{\beta'\gamma'} = -\frac{\partial^2 x^{\alpha'}}{\partial x^{\beta'} \partial x^{\lambda}} \frac{\partial x^{\lambda}}{\partial x^{\gamma'}} + \frac{\partial x^{\kappa}}{\partial x^{\beta'}} \frac{\partial x^{\lambda}}{\partial x^{\gamma'}} \frac{\partial x^{\alpha'}}{\partial x^{\rho}} \Gamma^{\rho}_{\kappa\,\lambda}$

```
Not Tensorial!
```

Notice, as promised, the connection coefficient exactly subtracts off the offending Hessian from the partial derivative of *A*!

Christoffel Symbols

- *A .wl block-version of code for this section is available at https://github.com/rebcabin/RelativityToolkit/blob/main/connections_to_christoffels.wl.*

Up to this point, we have not actually used a metric tensor! MTW devote seven whole chapters to differential geometry without a metric (9-15), stressing that many general results in differential geometry—the topological results—can be derived without the metric. In particular, the covariant derivative and the connection coefficients do not, actually, depend on any metrical ability to measure distances, but only on assumptions of continuity and differentiability.

But gravitation via general relativity requires a special case: Riemannian Geometry. For that, we need a metric, and even a bit more: symmetry conditions on both the connection coefficients and on the metric. From these additions, we derive the ultra-famous gravitational Christoffel symbols:

$$\Gamma^{\sigma}{}_{\mu\nu} = \frac{1}{2}\left(g^{\sigma\lambda}\right)\left(g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda}\right) \tag{9}$$

This is equation 8.24 of MTW, after renaming some indices. It is also called “The Fundamental Theorem of Riemannian Geometry” in mathematical texts like do Carmo and Lee (see References).

The following block of code shows a step-by-step derivation via $\mathcal{UD}$. As usual, the “prose” is in the comments and Echoes. Run it and read it!

In[•]:=
```
Print[
  "============================================================================="];
Print["Christoffel Symbols for Gravitation from Metric via the 𝒰𝒟 Calculus"];
Print[
  "============================================================================="];

ClearAll[g, GammaSymmetryRule, MetricSymmetryRule,
  term1, term2, term3, combination, targetIndex];

Print[
  "1. PHYSICAL CONSTRAINTS ----------------------------------------------------"];
Print["1a. Torsion-free geometry: Γ^α_βγ == Γ^α_γβ, [MTW] Box 10.2, page 250 "];
GammaSymmetryRule = Echo[
   Γ[u_, 𝒟[b_], 𝒟[c_]] :↦
    Γ[u, 𝒟[Sort[{b, c}][[1]]], 𝒟[Sort[{b, c}][[2]]]],
   "Γ Symmetry: "];

Print["1b. Metric Symmetry: g_αβ == g_βα, [MTW] Equation 8.24, page 210"];
Print[Style["(output manipulated as expected by 𝒰𝒟 display rules)", Gray]];
MetricSymmetryRule = Echo[
   g[𝒟[a_], 𝒟[b_]] :↦
    g[𝒟[Sort[{a, b}][[1]]], 𝒟[Sort[{a, b}][[2]]]],
   "g Symmetry"];

Print[
  "1c. Metric Compatibility Postulates ∇_[λ g_μν] = 0 [MTW] Eqn 8.23 & Chapter 14."];

(*Term 1: ∇_λ g_μν*)
term1 = CD[g[𝒟[μ], 𝒟[ν]], x[𝒰[λ]]] /. GammaSymmetryRule /. MetricSymmetryRule;
Echo[TensorForm[term1], "0 = ∇_λ g_μν ="];

(*Term 2: ∇_μ g_νλ (Left-Cycle Indices)*)
term2 = CD[g[𝒟[ν], 𝒟[λ]], x[𝒰[μ]]] /. GammaSymmetryRule /. MetricSymmetryRule;
Echo[TensorForm[term2], "0 = ∇_μ g_μλ ="];

(*Term 3: ∇_ν g_λμ (Left-Cycle Indices)*)
```

```
term3 = CD[g[𝒟[λ], 𝒟[μ]], x[𝒰[ν]]] /. GammaSymmetryRule /. MetricSymmetryRule;
Echo[TensorForm[term3], "0 = ∇ν gλμ ="];


Print[
  "2. CYCLIC PERMUTATION TRICK ---------------------------------------------"];
Print["Write 'Koszul' sum of ∇g permutations that isolates a Γ term."];
Print["[MTW] Equation 8.24b, page 210, in coordinate basis (cabc===0)"];
Print["The Linear Combination 0 = (∇μ gνλ + ∇ν gλμ - ∇λ gμν) ==="];

Echo[combination = (term2 + term3 - term1), "0 ="];

Echo[gammaPart = Select[List @@ combination, ! FreeQ[#, Γ] &] // Total,
  "Γ part ="];
Echo[gammaPartCanonical = gammaPart // CanonicalizeIndices,
  "Γ part (canonical) ="];
Echo[metricPartials = Select[List @@ combination, FreeQ[#, Γ] &] // Total,
  "g partials"];


Print[
  "3. SOLVE FOR Γ ------------------------------------------------------------"];

(*Equation: gammaPart + metricPartials = 0 => -gammaPart = metricPartials*)
Print["3a. Equation to solve:"];
lhsRaw = -gammaPartCanonical;
rhsRaw = metricPartials;

Echo[TensorForm[lhsRaw == rhsRaw], "Isolated Equation:"];

Print["3b. Find correct g^-1 to multiply through."];

(*1. Find the metric inside the LHS term*)
Echo[metricFound = First[Cases[lhsRaw, g[__], Infinity]], "g found"];

(*2. Find the Gamma inside the LHS term*)
Echo[gammaFound = First[Cases[lhsRaw, Γ[__], Infinity]], "Γ found"];

(*3. Extract raw symbols from the indices*)
Echo[metricIndices = First /@ (List @@ metricFound), "g indices"];
Echo[gammaUpIndex = First[First[gammaFound]], "Γ up index"];

(*4. Identify the Target Index*)
```

```
(*The g index that matches Γ up-index is the Dummy. The other is the Target.*)
Echo[targetIndex = If[metricIndices[[1]] === gammaUpIndex,
    metricIndices[[2]], metricIndices[[1]]], "target index"];

(*5. Construct the specific Inverse needed*)
Echo[invMetricFactor = 1 / 2 * g[𝒰[σ], 𝒰[targetIndex]], "1/2 g^-1 ="];
Print[Style["(1/2 cancels factor 2 from Γ part (canonical) above)", Gray]];

Print["3c. Multiply and Contract"];

(*δ-Γ contraction rule (will be promoted into the engine in next installment)*)
Echo[deltaContractionRule = δ[𝒰[s_], 𝒟[r_]] * Γ[𝒰[r_], a_, b_] :> Γ[𝒰[s], a, b],
  "δ-Γ contraction rule: "];
Print[Style["(will be promoted into 𝒰𝒟 in the future)", Gray]];

(*Execute the algebra*)
Echo[lhsSolved = lhsRaw * invMetricFactor //. metricRules //. deltaContractionRule,
  "LHS solved: "];

(*Format RHS:Factor out 1/2 g^σ_λ*)
Echo[rhsSolved = Collect[rhsRaw * invMetricFactor, g[𝒰[_], 𝒰[_]]],
  "RHS solved:"];

(*Step D: Final Result*)
Print["\n=== CHRISTOFFEL SYMBOLS FOR GRAVITATION ==="];
Print["aka The Fundamental Theorem of Riemannian Geometry"];
finalEquation = lhsSolved == rhsSolved;
TensorForm[finalEquation]
```

```
================================================================================
Christoffel Symbols for Gravitation from Metric via the 𝒰𝒟 Calculus
================================================================================
1. PHYSICAL CONSTRAINTS ----------------------------------------------------
1a. Torsion-free geometry: Γ^α_βγ == Γ^α_γβ, [MTW] Box 10.2, page 250
» Γ Symmetry: Γ[u_, 𝒟[b_], 𝒟[c_]] :> Γ[u, 𝒟[Sort[{b, c}][[1]]], 𝒟[Sort[{b, c}][[2]]]]
1b. Metric Symmetry: g_αβ == g_βα, [MTW] Equation 8.24, page 210
(output manipulated as expected by 𝒰𝒟 display rules)
» g Symmetry g a_ b_ :> g a b
1c. Metric Compatibility Postulates ∇_[λ g_μν] = 0 [MTW] Eqn 8.23 & Chapter 14.
» 0 = ∇_λ g_μν = g μ ν,λ - (g κ ν) Γ^κ_λμ - (g κ μ) Γ^κ_λν
» 0 = ∇_μ g_μλ = g λ ν,μ - (g κ ν) Γ^κ_λμ - (g λ κ) Γ^κ_μν
```

» $0 = \nabla_\nu g_{\lambda\mu} =$ g λ μ,ν − (g κ μ) Γ^κ_{λν} − (g λ κ) Γ^κ_{μν}

2. CYCLIC PERMUTATION TRICK ----------

Write 'Koszul' sum of ∇g permutations that isolates a Γ term.

[MTW] Equation 8.24b, page 210, in coordinate basis (c_{abc}===0)

The Linear Combination 0 = ($\nabla_\mu g_{\nu\lambda} + \nabla_\nu g_{\lambda\mu} - \nabla_\lambda g_{\mu\nu}$) ===

» 0 = g λ μ,ν + g λ ν,μ − g μ ν,λ + (g λ29 ν) Γ^{λ29}_{λμ} + (g λ30 μ) Γ^{λ30}_{λν} −
(g λ λ31) Γ^{λ31}_{μν} − (g λ32 ν) Γ^{λ32}_{λμ} − (g λ33 μ) Γ^{λ33}_{λν} − (g λ λ34) Γ^{λ34}_{μν}

» Γ part = (g λ29 ν) Γ^{λ29}_{λμ} + (g λ30 μ) Γ^{λ30}_{λν} − (g λ λ31) Γ^{λ31}_{μν} − (g λ32 ν) Γ^{λ32}_{λμ} − (g λ33 μ) Γ^{λ33}_{λν} − (g λ λ34) Γ^{λ34}_{μν}

» Γ part (canonical) = −2 (g λ i1) Γ^{i1}_{μν}

» g partials g λ μ,ν + g λ ν,μ − g μ ν,λ

3. SOLVE FOR Γ ----------

3a. Equation to solve:

» Isolated Equation: 2 (g λ κ) Γ^κ_{μν} == g λ μ,ν + g λ ν,μ − g μ ν,λ

3b. Find correct g^{-1} to multiply through.

» g found g λ i1

» Γ found Γ^{i1}_{μν}

» g indices {λ, i1}

» Γ up index i1

» target index λ

» $\frac{1}{2} g^{-1} =$ $\frac{1}{2}$ (g σ λ)

(1/2 cancels factor 2 from Γ part (canonical) above)

3c. Multiply and Contract

» δ–Γ contraction rule: Γ[𝒰[r_], a_, b_] $\delta^{s_-}_{r_-}$:↦ Γ[𝒰[s], a, b]

(will be promoted into 𝒰𝒟 in the future)

» LHS solved: $\Gamma^\sigma_{\mu\nu}$

» RHS solved: $\frac{1}{2}$ (g σ λ) (g λ μ,ν + g λ ν,μ − g μ ν,λ)

=== CHRISTOFFEL SYMBOLS FOR GRAVITATION ===

aka The Fundamental Theorem of Riemannian Geometry

Out[•]=

$$\Gamma^\sigma_{\mu\nu} == \frac{1}{2} (g^{\sigma\ \lambda}) (g_{\lambda\ \mu,\nu} + g_{\lambda\ \nu,\mu} - g_{\mu\ \nu,\lambda})$$

Conclusion

We began this notebook with a non-tensorial partial derivative of a contravariant tensor. However, by

adding a correction and *then* demanding linearity and tensoriality, we forced a covariant derivative to emerge, with the correction proportional to the non-tensorial connection Γ.

Also, not just accepting textbook derivations of the Christoffel symbols for gravitation, we derive them *ab-initio* in $\mathcal{UD}$. By insisting that lengths be preserved ($\nabla g = 0$) and twisting be forbidden ($\Gamma_{\mu\nu} = \Gamma_{\nu\mu}$), $\mathcal{UD}$ collapses the Koszul sum into the standard formula:

$$\Gamma^{\sigma}{}_{\mu\nu} = \frac{1}{2}\left(g^{\sigma\lambda}\right)\left(g_{\lambda\mu,\nu} + g_{\lambda\nu,\mu} - g_{\mu\nu,\lambda}\right)$$

The form of the Christoffel symbols shows that gravitation, manifested in the connection, is solely determined by the metric and its ordinary partial derivatives.

With a working covariant derivative in $\mathcal{UD}$'s Relativity Toolkit, we are equipped for future installments on geodesics, geodesic deviation, tidal forces, accelerated observers, Schwarzschild and Kerr metrics, black holes, etc.

References

1. Poisson, Eric. *A Relativist's Toolkit: The Mathematics of Black Hole Mechanics*. Cambridge University Press, 2004.
2. Beckman, Brian. *From the steam age to quantum fields: a unified approach via UD tensorial calculus*. Wolfram Community post.
3. Beckman, Brian. *General relativity in the UD calculus*. Wolfram Community post.
4. Beckman, Brian. *Formal Differential Geometry in the UD Calculus: Part 1*. Wolfram Community post.
5. [MTW] Misner, Charles W.; Thorne, Kip S.; Wheeler, John Archibald. *Gravitation*. Princeton University Press, 2017.
6. [L&R] Lovelock, David and Rund, Hanno. *Tensors, Differential Forms, and Variational Principles*. Dover.
7. John M. Lee, *Riemannian Manifolds: An Introduction to Curvature*. Springer.
8. Manfredo P. do Carmo, *Riemannian Geometry*. Springer.
9. Wolfram Function Repository. *MetricTensor, RiemannTensor, EinsteinTensor*. For component-based calculations.
10. Wolfram Research. *Mathematica Documentation.* Tensor Calculus.